

자바프로그래밍

구현 결과 보고서

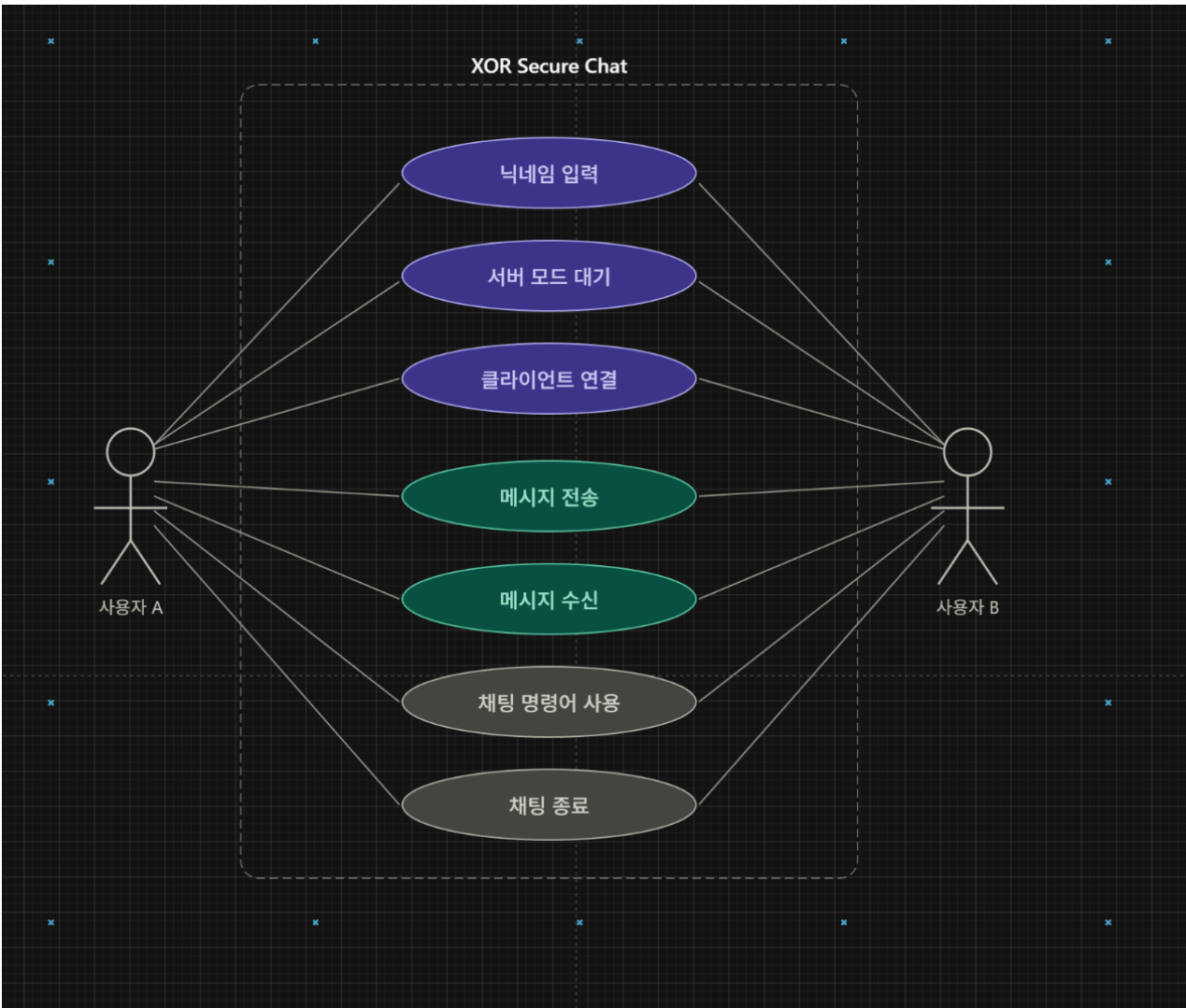
XOR 암호화 채팅 프로그램

1. 과제 개요

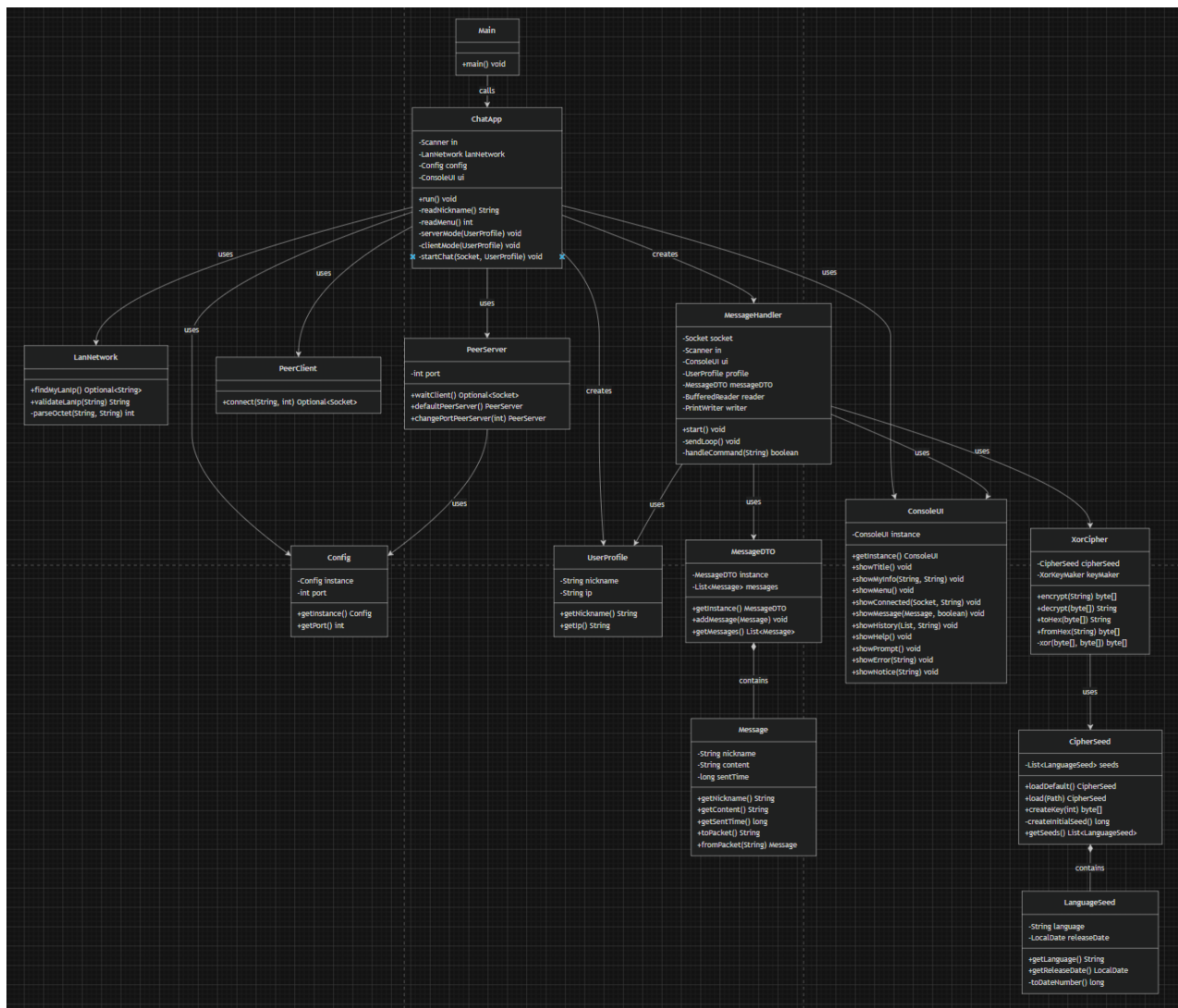
과제 제목	XOR 암호화 채팅 프로그램
주요 구현 목표	실시간 통신 및 채팅 데이터 암호화 복호화 구현
개발 환경	IntelliJ 2025.2.6.1
제출 파일	소스코드(.java), 본 보고서(.docx), AI 활용 보고서(.docx)

2. 클래스 설계

2.1 유스케이스 및 클래스 다이어그램



유스케이스(draw.io 사용)



클래스 다이어그램(draw.io 사용)

2.2 클래스 역할

클래스명	유형	역할
Main	실행 클래스	ChatApp.run()을 호출하여 프로그램을 시작하는 진입점
ChatApp	실행 클래스	앱 전체 흐름 제어. 닉네임 입력, 메뉴 선택, 서버/클라이언트 모드 분기 처리
Config	데이터 클래스	싱글톤 패턴으로 포트 번호 등 전역 설정값을 관리
PeerServer	네트워크 클래스	지정 포트에서 ServerSocket 을 열고 상대방의 TCP 접속을 대기
PeerClient	네트워크 클래스	상대방의 IP 와 포트로 Socket 연결을 시도
MessageHandler	통신 클래스	가상 스레드로 메시지 수신, sendLoop 로 메시지 송신을 처리

MessageDTO	데이터 클래스	싱글톤 패턴으로 채팅 메시지 목록을 저장하고 관리하는 DTO
XorCipher	암호화 클래스	XOR 연산으로 메시지를 암호화·복호화하고 16 진수 문자열로 변환
CipherSeed	데이터 클래스	시드 파일을 읽어 LCG 알고리즘 기반의 XOR 암호화 키를 생성
LanNetwork	네트워크 클래스	로컬 LAN IP 를 탐색하고 입력된 IP 주소의 유효성을 검증
UserProfile	데이터 클래스	닉네임과 IP 주소로 구성된 사용자 프로필 정보를 보관
ConsoleUI	UI 클래스	싱글톤 패턴으로 콘솔 출력을 전담. 메뉴, 메시지, 연결 정보 등을 출력

2.3 인터페이스 및 구현클래스

※ 인터페이스 및 구현 클래스로 어떻게 정의했는지 기술하세요.

인터페이스	구현 클래스	역할
XorKeyMaker	XorCipher	XOR 키 생성 전략을 정의하는 함수형 인터페이스. makeKey(int) 메서드로 암호화 키를 생성
-	-	-
-	-	-

3. 주요 구현 내용

3.1 핵심 기능 구현 현황

※ 핵심 기능별 구현 클래스 및 메소드를 자유롭게 기술하세요.

No	기능	구현 클래스	메소드	설명
1	XOR 암호화 / 복호화	XorCipher	encrypt(), decrypt()	LCG 기반 키로 메시지를 XOR 암호화하고 16 진수로 변환하여 전송
2	암호화 키 생성	CipherSeed	createKey(), createInitialSeed()	시드 파일의 언어별 날짜를 조합해 LCG

				알고리즘으로 XOR 키를 생성
3	P2P 서버 대기	PeerServer	waitClient()	지정 포트로 ServerSocket 을 열고 상대방의 TCP 연결을 대기
4	P2P 클라이언트 접속	PeerClient	connect()	상대방의 IP 와 포트로 Socket 접속을 시도하고 결과를 Optional 로 반환
5	메시지 송신	MessageHandler	sendLoop()	사용자 입력을 받아 암호화 후 상대방에게 전송, 명령어(/quit 등) 처리
6	메시지 수신	MessageHandler	ReceiveTask.run()	가상 스레드에서 상대방 메시지를 수신. 복호화하여 콘솔에 출력
7	LAN IP 탐지 및 검증	LanNetwork	findMyLanIp(), validateLanIp()	본인의 LAN IP 를 자동 탐지하고 상대방 입력 IP 의 형식 및 범위 유효성 검증
8	채팅 기록 및 명령어 처리	MessageHandler, MessageDTO	handleCommand(), addMessage()	/history, /info, /clear, /quit 등 명령어 처리 및 메시지 내역 관리

3.2 주요 코드 설명

※ 클래스별 중요한 메서드 2개 이상을 선택해 코드와 설명을 작성하세요.

▶ 메서드 1: serverMode() - ChatApp

```
private static void serverMode(UserProfile profile) {    ui.showWaiting(config.getPort());    PeerServer server =
```

```
PeerServer.defaultPeerServer();    server.waitClient().ifPresentOrElse(        socket -> startChat(socket,
profile),        () -> ui.showError("상대방과 연결하지 못했습니다.")    ); }
```

설명: 서버 모드 진입 시 *PeerServer* 를 생성하고 *waitClient()*로 상대방 연결을 대기한다. 연결 결과를 *Optional* 로 받아 *ifPresentOrElse()*로 처리해, 연결 성공 시 *startChat()*을 호출하고 실패 시 오류 메시지를 출력하는 분기 구조다. *Optional* 을 활용해 null 체크 없이 안전하게 연결 성공·실패를 처리하는 점이 핵심이다.

▶ 메서드 2: *readNickname()* - *ChatApp*

```
private static String readNickname() {    while (true) {        System.out.print("닉네임을 입력해주세요 > ");
String nickname = in.nextLine().trim();        if (!nickname.isEmpty()) {            return nickname;        }
ui.showError("닉네임은 비워둘 수 없습니다.");    } }
```

설명: 사용자로부터 닉네임을 입력받는 루프 메서드. 빈 문자열이 입력되면 오류 메시지를 출력하고 다시 입력을 요구하는 방어적 설계를 갖추고 있다. 유효한 입력이 들어올 때까지 루프를 반복하므로, 닉네임이 반드시 비어있지 않다는 불변식을 보장하고 그 값을 *UserProfile* 생성에 사용할 수 있게 한다.

▶ 메서드 3: *getInstance()* - *Config*

```
public static Config getInstance() {    return instance; }
```

설명: 싱글톤 패턴으로 구현된 *Config* 인스턴스 반환 메서드. 클래스 로드 시점에 단 하나의 인스턴스를 생성해 *static* 필드에 저장하고, *getInstance()*를 통해서만 접근 가능하도록 생성자를 *private* 으로 막아두었다. 포트 번호와 같은 전역 설정값이 프로그램 전체에서 하나의 객체로 일관성 있게 관리된다.

▶ 메서드 4: *getPort()* - *Config*

```
public int getPort() {    return port; }
```

설명: 서버와 클라이언트가 공통으로 사용할 포트 번호(5000)를 반환하는 메서드. *Config* 인스턴스를 통해서만 포트 번호를 얻을 수 있게 하여, 매직 넘버가 코드 곳곳에 하드코딩되는 것을 막고 설정 변경 시 한 곳만 수정하면 되는 유지보수성을 확보한다.

▶ 메서드 5: *waitClient()* - *PeerServer*

```
public Optional<Socket> waitClient() {    try {        ServerSocket serverSocket = new ServerSocket(port);
System.out.println("[서버 대기] PORT: " + port);        Socket socket = serverSocket.accept();
System.out.println("[연결됨] " + socket.getInetAddress().getHostAddress());        return Optional.of(socket);
} catch (IOException e) {        System.out.println("서버 오류: " + e.getMessage());        return
Optional.empty();    } }
```

설명: 지정 포트로 *ServerSocket* 을 열고 *accept()*로 상대방의 TCP 연결을 블로킹 대기하는 메서드. 연결에 성공하면 *Socket* 을 *Optional* 로 감싸 반환하고, *IOException* 발생 시 *Optional.empty()*를 반환해 호출부가 연결 실패를 null 없이 안전하게 처리할 수 있도록 한다.

▶ 메서드 6: *changePortPeerServer()* - *PeerServer*

```
public static PeerServer changePortPeerServer(int port) {    return new PeerServer(port); }
```

설명: 기본 포트 대신 사용자가 지정한 포트 번호로 *PeerServer* 인스턴스를 생성하는 정적 팩터리 메서드. 생성자를 직접 호출하지 않고 의미 있는 이름의 팩터리 메서드를 통해 객체를 생성하므로, 포트 변경 의도를 코드 읽는 사람이 즉시 파악할 수 있어 가독성이 높다.

▶ 메서드 7: *connect()* - *PeerClient*

```
public Optional<Socket> connect(String ip, int port) {
    System.out.println("[접속 성공] " + ip + ":" + port);
    System.out.println("접속 실패: " + e.getMessage());
}

try {
    Socket socket = new Socket(ip, port);
    return Optional.of(socket);
} catch (Exception e) {
    return Optional.empty();
}
```

설명: 상대방의 IP와 포트로 Socket 연결을 시도하고 결과를 Optional로 반환하는 메서드. 연결 성공 시 *Optional.of(socket)*, 실패 시 *Optional.empty()*를 반환해 예외 없이 성공·실패를 처리할 수 있도록 설계했다. *catch* 블록에서 *IOException* 대신 *Exception*을 잡아 소켓 생성 과정에서 발생할 수 있는 예상치 못한 예외도 안전하게 처리한다.

▶ 메서드 8: *start()* - *MessageHandler*

```
public void start() throws IOException {
    ExecutorService executor =
    Executors.newVirtualThreadPerTaskExecutor();
    sendLoop();
} finally {
    socket.close();
    executor.shutdownNow();
}

try {
    executor.submit(new ReceiveTask());
}
```

설명: 채팅 송수신을 시작하는 메서드다. 가상 스레드를 하나 만들어서 메시지 받는 일을 맡기고, 메인 스레드는 메시지 보내는 일을 담당한다. 채팅이 끝나면 *finally* 블록에서 소켓과 스레드를 반드시 정리해서 자원이 낭비되지 않도록 한다.

▶ 메서드 9: *sendLoop()* - *MessageHandler*

```
private void sendLoop() {
    while (!socket.isClosed()) {
        in.nextLine().trim();
        if (socket.isClosed()) { break; }
        if (content.isEmpty()) { continue; }
        if (handleCommand(content)) {
            if ("/quit".equals(content)) { break; }
            continue;
        }
        MessageDTO.Message message = new MessageDTO.Message(profile.getNickname(), content);
        messageDTO.addMessage(message);
        String enc =
        XorCipher.toHex(XorCipher.encrypt(message.toPacket()));
        writer.println(enc);
        ui.showMessage(message, true);
    }
}
```

설명: 메인 스레드에서 동작하며 사용자 입력을 읽어 처리하는 루프 메서드. 입력이 빈 문자열이면 무시하고, '/'로 시작하는 명령어면 *handleCommand()*로 분기하며, 일반 메시지면 *XorCipher*로 암호화해 *PrintWriter*로 전송한다. */quit* 입력 시 루프를 종료하고 소켓을 닫아 *ReceiveTask*도 자연스럽게 종료되도록 흐름을 설계했다.

▶ 메서드 10: *handleCommand()* - *MessageHandler*

```
private boolean handleCommand(String content) {
    switch (content) {
        case "/help" -> ui.showHelp();
        case "/history" -> ui.showHistory(messageDTO.getMessages(), profile.getNickname());
        case "/info" -> ui.showConnectionInfo(socket, profile.getNickname());
        case "/clear" -> { ui.clear();
        ui.showConnected(socket, profile.getNickname()); }
        case "/quit" -> ui.showNotice("채팅을 종료합니다.");
        default -> { return false; }
    }
    return true;
}
```

설명: 사용자가 입력한 명령어 문자열을 *switch*로 분기해 */help*, */history*, */info*, */clear*, */quit*를 각각 처리하는 메서드. 명령어를 처리했으면 *true*, 일반 메시지면 *false*를 반환해 *sendLoop()*에서 메시지 전송 여부를 결정하게 한다. 명령어 로직을 별도 메서드로 분리함으로써 *sendLoop()*의 복잡도를 낮추고 각 명령어를 독립적으로 확장하기 쉬운 구조를 갖추고 있다.

▶ 메서드 11: *addMessage()* - *MessageDTO*

```
public void addMessage(Message message) {
    if (message == null) {
        throw new
        IllegalArgumentException("저장할 메시지가 없습니다.");
    }
    messages.add(message);
}
```

설명: 채팅 메시지를 내부 리스트에 추가하는 메서드. *null* 이 전달될 경우 *IllegalArgumentException* 을 던져 잘못된 데이터가 저장소에 들어오는 것을 원천 차단한다. 싱글톤 인스턴스의 메시지 리스트에 누적 저장되므로 */history* 명령어 실행 시 세션 내 전체 메시지 이력을 복원할 수 있다.

▶ 메서드 12: *fromPacket()* - *MessageDTO.Message*

```
public static Message fromPacket(String packet) { if (packet == null || packet.isBlank()) { throw new
IllegalArgumentException("메시지 패킷이 비어 있습니다."); } String[] parts = packet.split("Wwt", 3); if
(parts.length != 3) { throw new IllegalArgumentException("메시지 패킷 형식이 올바르지 않습니다."); }
long sentTime = Long.parseLong(parts[0]); return new Message(parts[1], parts[2], sentTime); }
```

설명: 탭 문자('Wwt')로 구분된 패킷 문자열을 파싱해 *Message* 객체로 복원하는 정적 팩터리 메서드. *split("Wwt", 3)*으로 최대 3 개 토큰으로 나눠 *sentTime*, *nickname*, *content* 를 추출하고, 파싱 실패 시 명확한 메시지의 예외를 던진다. *toPacket()*과 대칭을 이루어 메시지 직렬화·역직렬화를 완결하는 구조다.

▶ 메서드 13: *encrypt()* / *decrypt()* - *XorCipher*

```
public static byte[] encrypt(String text) { byte[] data = text.getBytes(StandardCharsets.UTF_8); return
xor(data, keyMaker.makeKey(data.length)); } public static String decrypt(byte[] data) { byte[] result =
xor(data, keyMaker.makeKey(data.length)); return new String(result, StandardCharsets.UTF_8); }
```

설명: 메시지를 암호화하고 복호화하는 메서드다. 텍스트를 바이트 배열로 바꾼 뒤 같은 길이의 키와 XOR 연산을 수행한다. XOR은 같은 키로 두 번 연산하면 원래 값으로 돌아오는 성질이 있어서 암호화와 복호화가 완전히 같은 방식으로 동작한다.

▶ 메서드 14: *toHex()* / *fromHex()* - *XorCipher*

```
public static String toHex(byte[] data) { StringBuilder sb = new StringBuilder(); for (byte b : data) {
sb.append(String.format("%02X", b)); } return sb.toString(); } public static byte[] fromHex(String hex) {
if (hex == null || hex.isBlank() || hex.length() % 2 != 0) { throw new IllegalArgumentException("16 진수
암호문 형식이 올바르지 않습니다."); } byte[] data = new byte[hex.length() / 2]; for (int i = 0; i <
data.length; i++) { int pos = i * 2; data[i] = (byte) Integer.parseInt(hex.substring(pos, pos + 2), 16);
} return data; }
```

설명: 암호화된 바이트 배열을 16 진수 문자열로, 혹은 그 반대로 변환하는 유틸리티 메서드 쌍. 소켓 통신은 텍스트 기반(PrintWriter/BufferedReader)이므로 바이트 배열을 그대로 전송할 수 없는데, 16 진수 문자열로 변환하면 모든 바이트값을 손실 없이 ASCII 문자로 전달할 수 있다. *fromHex()*는 홀수 길이 입력을 사전 검증해 파싱 오류를 방지한다.

▶ 메서드 15: *createKey()* - *CipherSeed*

```
public byte[] createKey(int length) { if (length < 1) { throw new IllegalArgumentException("키 길이는
1 이상이어야 합니다."); } byte[] key = new byte[length]; long value = createInitialSeed(); for (int i
= 0; i < key.length; i++) { value = (value * A + C) % M; key[i] = (byte) value; } return key; }
```

설명: XOR 암호화에 쓸 키를 만드는 메서드다. *createInitialSeed()*로 초기 시드값을 구한 뒤 A, C, M 값을 이용한 계산을 반복해서 키 바이트를 하나씩 만들어낸다. 양쪽이 같은 시드 파일을 갖고 있어야만 같은 키가 만들어지기 때문에 시드 파일이 다르면 복호화가 안 된다.

▶ 메서드 16: *load()* - *CipherSeed*

```
public static CipherSeed load(Path path) { List<LanguageSeed> seeds = new ArrayList<>(); Set<String>
```

```
languageNames = new HashSet<>(); try (BufferedReader reader = Files.newBufferedReader(path,
StandardCharsets.UTF_8)) { String line; int lineNumber = 0; while ((line = reader.readLine()) !=
null) { lineNumber++; line = line.trim(); if (line.isEmpty() || line.startsWith("#"))
{ continue; } String[] parts = line.split("=", 2); String language =
parts[0].trim().toUpperCase(); LocalDate releaseDate = LocalDate.parse(parts[1].trim()); if (!
languageNames.add(language)) { throw new IllegalArgumentException("중복된 언어 시드입니다: "
+ language); } seeds.add(new LanguageSeed(language, releaseDate)); } } catch
(IOException e) { throw new IllegalStateException("시드 파일을 읽을 수 없습니다: " + path, e); }
return new CipherSeed(seeds); }
```

설명: 지정 경로의 시드 파일을 읽어 CipherSeed 객체를 생성하는 정적 팩터리 메서드. 각 줄을 '=' 구분자로 파싱해 언어명과 날짜를 추출하고, 중복 언어명이나 잘못된 날짜 형식에 대해 IllegalArgumentException 을 던진다. 파일 입출력 예외는 IllegalStateException 으로 래핑해 호출부가 체크 예외 없이 처리할 수 있게 한다.

▶ 메서드 17: createInitialSeed() - CipherSeed

```
private long createInitialSeed() { long initialSeed = 0; for (int i = 0; i < seeds.size(); i++) { long
dateNumber = seeds.get(i).toDateNumber(); initialSeed += dateNumber * (i + 1L); } return
initialSeed; }
```

설명: 시드 파일에 등록된 각 언어의 출시일을 YYYYMMDD 형태의 숫자로 변환한 뒤, 인덱스 가중치(i+1)를 곱해 합산하여 초기 시드값을 만드는 메서드. 언어 순서가 달라지면 가중치 합산 결과가 달라져 완전히 다른 키가 생성되므로, 시드 파일의 순서까지 일치해야만 정상적으로 복호화가 가능한 구조를 만들어낸다.

▶ 메서드 18: validateLanIp() - LanNetwork

```
public String validateLanIp(String ip) { if (ip == null || ip.isBlank()) { throw new
IllegalArgumentException("상대 IP가 필요합니다."); } String[] parts = ip.trim().split("www."); if
(parts.length != 4) { throw new IllegalArgumentException("IP는 192.168.200.188 형식으로
입력해주세요."); } int first = parseOctet(parts[0], "첫 번째"); int second = parseOctet(parts[1], "두
번째"); int third = parseOctet(parts[2], "세 번째"); int fourth = parseOctet(parts[3], "네 번째"); if
(first != 192 || second != 168) { throw new IllegalArgumentException("192.168.x.x LAN IP만 연결할 수
있습니다."); } return first + "." + second + "." + third + "." + fourth; }
```

설명: 입력받은 IP 주소가 올바른지 확인하는 메서드다. 점을 기준으로 4개로 나뉘서 각 부분이 숫자인지, 0~255 범위인지, 192.168로 시작하는지를 순서대로 검사한다. 어느 부분에서 틀렸는지 구체적인 오류 메시지를 보내주기 때문에 사용자가 뭐가 잘못됐는지 바로 알 수 있다.

▶ 메서드 19: findMyLanIp() - LanNetwork

```
public Optional<String> findMyLanIp() { try { return
Collections.list(NetworkInterface.getNetworkInterfaces()).stream() .filter(ni -> { try
{ return ni.isUp() && !ni.isLoopback() && !ni.isVirtual(); } catch (SocketException e) { return
false; } }) .flatMap(ni -> Collections.list(ni.getInetAddresses()).stream() .filter(addr -> addr instanceof Inet4Address)
.map(addr -> addr.getHostAddress()) .filter(ip -> ip.startsWith("192.168.")).findFirst();
} catch (SocketException e) { return Optional.empty(); } }
```

설명: 시스템의 모든 네트워크 인터페이스를 스트림으로 탐색해 활성화된 비루프백, 비가상 인터페이스 중 192.168로 시작하는 IPv4 주소를 찾아 Optional로 반환하는 메서드. 스트림과 filter, flatMap, findFirst를 연계해 네트워크 인터페이스 열거 코드를 간결하게 표현했으며, LAN IP를 찾지 못해도 예외 없이 Optional.empty()로 처리한다.

▶ 메서드 20: getNickname() / getIp() - UserProfile

```
public String getNickname() { return nickname; } public String getIp() { return ip; }
```

설명: 사용자 닉네임과 IP 주소를 반환하는 접근자 메서드. *UserProfile*은 생성자에서 두 값 모두 *null* 또는 공백 여부를 검증하고 불변 필드로 저장하기 때문에, 이 메서드들이 반환하는 값은 항상 유효한 문자열임이 보장된다. 캡슐화를 통해 외부에서 프로필 정보를 직접 수정할 수 없도록 막는 역할도 한다.

▶ 메서드 21: *showMessage()* - *ConsoleUI*

```
public void showMessage(MessageDTO.Message message, boolean mine) { String time =  
TIME_FORMAT.format(Instant.ofEpochMilli(message.getSentTime())); String sender = mine ? "나" :  
message.getNickname(); System.out.println(); System.out.println "[" + time + "]" + sender + "> " +  
message.getContent()); }
```

설명: 수신된 *Message* 객체를 포맷에 맞춰 콘솔에 출력하는 메서드. *sentTime*을 *Instant*로 변환해 *HH:mm* 형식의 시간을 구하고, *mine* 플래그에 따라 발신자 표시를 '나' 또는 상대방 닉네임으로 구분한다. UI 출력 로직을 *ConsoleUI*에 집중시켜 *MessageHandler*가 출력 방식에 의존하지 않도록 관심사를 분리했다.

▶ 메서드 22: *showConnected()* - *ConsoleUI*

```
public void showConnected(Socket socket, String nickname) { clear(); System.out.println(LINE);  
System.out.println(" " + nickname + "의 XOR SECURE CHAT"); System.out.println("상태: " +  
socket.getInetAddress().getHostAddress() + "와 연결됨"); System.out.println(LINE);  
showCommandLine(); }
```

설명: 채팅방 연결이 성립됐을 때 화면을 초기화하고 연결 정보를 출력하는 메서드. *clear()*를 먼저 호출해 이전 메뉴 화면을 지우고 닉네임과 상대방 IP를 헤더로 표시한 뒤 명령어 안내를 함께 출력한다. 연결 직후 사용자가 채팅 환경에 바로 집중할 수 있도록 UI 상태를 일관되게 초기화하는 역할을 담당한다.

4. 구현 결과

4.1 실행 결과 및 설명

```
=====
XOR SECURE CHAT
=====
닉네임을 입력해주세요 >
```

최초 프로그램 실행 시 띄워지는 화면이며 닉네임 입력 창이 나옵니다.

```
=====
XOR SECURE CHAT
=====
닉네임을 입력해주세요 > 나병준1

[내 정보]
닉네임 : 나병준1
IP      : 192.168.200.125
-----
1. 상대방의 연결 기다리기
2. 상대방에게 연결하기
0. 프로그램 종료
-----
메뉴 선택 > |
```

닉네임 입력 후 본인의 IP 주소를 출력해주며 상대가 접속해오게 연결을 기다릴지 아니면 기다리는 상대에게 연결을 할지 중에 선택을 합니다.

```
=====
                        XOR SECURE CHAT
=====
닉네임을 입력해주세요 > 나병준1

[내 정보]
닉네임 : 나병준1
IP      : 192.168.200.125
-----
1. 상대방의 연결 기다리기
2. 상대방에게 연결하기
0. 프로그램 종료
-----
메뉴 선택 > 1

[연결 대기 중] PORT: 5000
상대방의 연결을 기다리고 있습니다...
[서버 대기] PORT: 5000
```

연결을 기다리는 쪽에서는 포트 번호가 나오며 상대의 연결을 기다리고 있다는 출력이 나옵니다.

```
=====
                        XOR SECURE CHAT
=====
닉네임을 입력해주세요 > 나병준2

[내 정보]
닉네임 : 나병준2
IP      : 192.168.200.188
-----
1. 상대방의 연결 기다리기
2. 상대방에게 연결하기
0. 프로그램 종료
-----
메뉴 선택 > 2
상대 IP 입력(예: 192.168.200.188) > |
```

연결을 하는 쪽에서는 상대의 IP를 입력하라는 창이 나오며

```
=====
                        XOR SECURE CHAT
=====
닉네임을 입력해주세요 > 나병준2

[내 정보]
닉네임 : 나병준2
IP      : 192.168.200.188
-----

1. 상대방의 연결 기다리기
2. 상대방에게 연결하기
0. 프로그램 종료
-----

메뉴 선택 > 2
상대 IP 입력(예: 192.168.200.188) > 192.168.200.125
[접속 성공] 192.168.200.125:5000
=====
                        나병준2의 XOR SECURE CHAT
상태: 192.168.200.125와 연결됨
=====
/help | /history | /info | /clear | /quit
-----

메시지 입력 > |
```

상대방의 IP 를 입력 시 채팅방이 연결되게 됩니다.

```
=====
                                XOR SECURE CHAT
=====
닉네임을 입력해주세요 > 나병준1

[내 정보]
닉네임 : 나병준1
IP      : 192.168.200.125
-----
1. 상대방의 연결 기다리기
2. 상대방에게 연결하기
0. 프로그램 종료
-----
메뉴 선택 > 1

[연결 대기 중] PORT: 5000
상대방의 연결을 기다리고 있습니다...
[서버 대기] PORT: 5000
[연결됨] 192.168.200.188
=====
                                나병준1의 XOR SECURE CHAT
상태: 192.168.200.188와 연결됨
=====
/help | /history | /info | /clear | /quit
-----
메시지 입력 > |
```

상대방의 시점에서도 연결된 모습을 볼 수 있습니다.

```
=====
                        나병준1의 XOR SECURE CHAT
상태: 192.168.200.188와 연결됨
=====
/help | /history | /info | /clear | /quit
-----

메시지 입력 > 안녕하세요!

[00:20] 나 > 안녕하세요!
메시지 입력 >
[00:20] 나병준2 > 안녕하세요!
메시지 입력 > Hello Java

[00:21] 나 > Hello Java
메시지 입력 >
[00:21] 나병준2 > Hello Java
메시지 입력 > |
```

[00:22] 나 > 1234567890 !@#\$%

메시지 입력 >

[00:22] 나병준2 > 1234567890 !@#\$%

메시지 입력 >

[00:23] 나병준2 > 동해물과 백두산이 마르고 닳도록 하느님이 보우하사 우리 나라 만세 무궁화 삼천리 화려강산 대한 사람 대한으로 길이 보전하세 남산위에 저 소나무 철갑을 두른 듯 바람서리 불변함은 우리 기상일세 무궁화 삼천리 화려강산 대한 사람 대한으로 길이 보전하세

메시지 입력 > 동해물과 백두산이 마르고 닳도록 하느님이 보우하사 우리 나라 만세 무궁화 삼천리 화려강산 대한 사람 대한으로 길이 보전하세 남산위에 저 소나무 철갑을 두른 듯 바람서리 불변함은 우리 기상일세 무궁화 삼천리 화려강산 대한 사람 대한으로 길이 보전하세

[00:24] 나 > 동해물과 백두산이 마르고 닳도록 하느님이 보우하사 우리 나라 만세 무궁화 삼천리 화려강산 대한 사람 대한으로 길이 보전하세 남산위에 저 소나무 철갑을 두른 듯 바람서리 불변함은 우리 기상일세 무궁화 삼천리 화려강산 대한 사람 대한으로 길이 보전하세

메시지 입력 > |

위 사진과 같이 한글, 영어, 긴 문자, 특수 문자 모두 암호화 및 복호화 되어 정상적으로 채팅창에 띄워지는 것을 확인해 볼 수 있습니다.

```
메시지 입력 > /help
```

```
-----
/help    사용 가능한 명령어 출력
/history 저장된 채팅 기록 출력
/info    현재 연결 정보 출력
/clear   터미널 화면 정리
/quit    채팅 종료
-----
```

메시지 창에서 확인해 볼 수 있는 명령어 모음을 띄우는 명령어 입니다.

```
메시지 입력 > /history
```

```
-----
[채팅 기록]
[00:20] 나 > 안녕하세요!
[00:20] 나병준2 > 안녕하세요!
[00:21] 나 > Hello Java
[00:21] 나병준2 > Hello Java
[00:22] 나 > 1234567890 !@#%$
[00:22] 나병준2 > 1234567890 !@#%$
[00:23] 나병준2 > 동해물과 백두산이 마르고 닳도록 하느님이 보우하사 우리 나라 만세 무궁화 삼천리 화려강산 대한 사람 대한으로 길이 보전하세 남산위에 저 소나무 철갑을 두른 듯 바람서리 불변함은 우리 기상일세 무궁화 삼천리 화려강산 대한 사람 대한으로 길이 보전하세
[00:24] 나 > 동해물과 백두산이 마르고 닳도록 하느님이 보우하사 우리 나라 만세 무궁화 삼천리 화려강산 대한 사람 대한으로 길이 보전하세 남산위에 저 소나무 철갑을 두른 듯 바람서리 불변함은 우리 기상일세 무궁화 삼천리 화려강산 대한 사람 대한으로 길이 보전하세
-----
```

/history 를 입력하게 되면 지금까지 대화했던 로그를 전체적으로 띄워서 확인해볼 수 있습니다.

```
메시지 입력 > /info
```

```
-----
[현재 연결 정보]
닉네임    : 나병준1
내 주소   : 192.168.200.125
상대 주소 : 192.168.200.188
포트      : 5000
-----
```

```
메시지 입력 > |
```

/info 를 입력하게 되면 현재 닉네임, 본인과 상대방의 IP 주소 채팅방의 연결 포트 번호를 확인해볼 수 있습니다.

```

메시지 입력 > /clear
=====
                        나병준1의 XOR SECURE CHAT
상태: 192.168.200.188와 연결됨
=====
/help | /history | /info | /clear | /quit
-----
메시지 입력 > |

```

/clear 를 치게 되면 다시 초기 채팅 연결 화면을 띄워 화면을 정리해 줍니다.

```

=====
                        나병준1의 XOR SECURE CHAT
상태: 192.168.200.188와 연결됨
=====
/help | /history | /info | /clear | /quit
-----
메시지 입력 > /quit
[알림] 채팅을 종료합니다.

종료 코드 0(으)로 완료된 프로세스

```

```

-----
메시지 입력 > [알림] 상대방이 채팅을 종료했습니다.

```

/quit 를 치면 채팅을 종료하게 됩니다. 상대방에게도 상대방이 채팅을 종료하였다는 코멘트를 띄워주며 프로그램을 종료 시킵니다.

여기서 부터는 각종 예외처리들을 다루겠습니다.

```

=====
                        XOR SECURE CHAT
=====
닉네임을 입력해주세요 >
[오류] 닉네임은 비워둘 수 없습니다.

```

닉네임에 공백을 입력하게 되면 위와 같이 오류 메시지를 출력합니다.

```
=====
                                XOR SECURE CHAT
=====

닉네임을 입력해주세요 >
[오류] 닉네임은 비워둘 수 없습니다.
닉네임을 입력해주세요 > 나병준1

[내 정보]
닉네임 : 나병준1
IP      : 192.168.200.125
-----

1. 상대방의 연결 기다리기
2. 상대방에게 연결하기
0. 프로그램 종료
-----

메뉴 선택 > 3
[오류] 0, 1, 2 중 하나를 입력해주세요.
1. 상대방의 연결 기다리기
2. 상대방에게 연결하기
0. 프로그램 종료
-----

메뉴 선택 > asdf
[오류] 0, 1, 2 중 하나를 입력해주세요.
1. 상대방의 연결 기다리기
2. 상대방에게 연결하기
0. 프로그램 종료
-----
```

상대방의 연결을 기다릴지 아니면 상대방에게 연결을 할지 정하는 메뉴에서 0, 1, 2 번을 선택하지 않거나 다른 문자를 넣게 되면 위와 같이 오류 메시지를 출력합니다.

```
=====
                                XOR SECURE CHAT
=====
닉네임을 입력해주세요 > 나병준1

[내 정보]
닉네임 : 나병준1
IP      : 192.168.200.125
-----
1. 상대방의 연결 기다리기
2. 상대방에게 연결하기
0. 프로그램 종료
-----
메뉴 선택 > 2
상대 IP 입력(예: 192.168.200.188) > 192.168.999.1
[오류] 세 번째 IP 구역은 0부터 255 사이여야 합니다.

종료 코드 0(으)로 완료된 프로세스
```

```
=====
                                XOR SECURE CHAT
=====
닉네임을 입력해주세요 > 나병준1

[내 정보]
닉네임 : 나병준1
IP      : 192.168.200.125
-----
1. 상대방의 연결 기다리기
2. 상대방에게 연결하기
0. 프로그램 종료
-----
메뉴 선택 > 2
상대 IP 입력(예: 192.168.200.188) > 192.168.1
[오류] IP는 192.168.200.188 형식으로 입력해주세요.

종료 코드 0(으)로 완료된 프로세스
```

```

=====
                        XOR SECURE CHAT
=====
닉네임을 입력해주세요 > 나병준1

[내 정보]
닉네임 : 나병준1
IP      : 192.168.200.125
-----
1. 상대방의 연결 기다리기
2. 상대방에게 연결하기
0. 프로그램 종료
-----
메뉴 선택 > 2
상대 IP 입력(예: 192.168.200.188) > 192.168.200.123
접속 실패: Connection timed out: connect
[오류] 상대방에게 연결하지 못했습니다.

종료 코드 0(으)로 완료된 프로세스

```

상대방에게 연결하기를 선택한 후 IP 구역을 잘못 입력하거나 올바르지 않은 형태를 입력하거나 마지막으로 잘못된 IP 주소를 입력했을 시 각각 위 사진과 같은 오류 메시지를 띄우며 프로그램을 종료합니다.

아래 2 개의 사진은 txt 파일로 저장해둔 시드값이 다른 경우입니다.

```

=====
                        나병준1의 XOR SECURE CHAT
상태: 192.168.200.188와 연결됨
=====
/help | /history | /info | /clear | /quit
-----
메시지 입력 > 안녕하세요?

[00:34] 나 > 안녕하세요?
메시지 입력 > [오류] 메시지 변환 오류: 메시지 패킷 형식이 올바르지 않습니다.

```

```
=====
                        나병준2의 XOR SECURE CHAT
상태: 192.168.200.125와 연결됨
=====
/help | /history | /info | /clear | /quit
-----
메시지 입력 > [오류] 메시지 변환 오류: 메시지 패킷 형식이 올바르지 않습니다.
안녕하세요?

[00:34] 나 > 안녕하세요?
메시지 입력 > /history

-----
[채팅 기록]
[00:34] 나 > 안녕하세요?
-----
메시지 입력 > |
```

시드값이 다른 경우에서 서로 통신할 시 암호화 복호화 과정에서 키 값이 맞지 않아 메시지 패킷 형식이 올바르지 않다는 오류 메시지를 띄우며 상대가 무슨 입력을 했는지 확인하지 못하게 됩니다. /history 를 입력해봐도 본인이 입력한 문자 밖에 뜨지 않는 것을 확인할 수 있습니다.

시드값은 이런식으로 설정하였습니다.

1	JAVA=1995-05-23
2	PYTHON=1991-02-20
3	KOTLIN=2011-07-22
4	RUST=2015-05-15

아래 사진은 시드값의 순서를 바꿔 두 시드값이 다른 상황을 테스트 할 때의 한 쪽 시드값입니다.

1	RUST=2015-05-15
2	JAVA=1995-05-23
3	PYTHON=1991-02-20
4	KOTLIN=2011-07-22

5. 구현 과정 회고

5.1 수업 후기

우선 자바라는 언어의 문법뿐만 아니라 OOP 전체의 틀과 기초를 잡을 수 있었습니다. 중간고사 전까지는 for-each 문이나 새로운 switch 문 같은 최신 문법들을 배웠다면, 기말고사까지는 상속, 캡슐화, 다형성, 추상화라는 OOP 4대 핵심 개념을 명확히 이해하고 실제 프로그래밍에 어떻게 활용하는지 배울 수 있었습니다. 스트림, 람다, Optional 등 함수형 프로그래밍 요소들의 활용법,

직렬화·역직렬화, 디자인 패턴, 그리고 이번 프로젝트에서 직접 사용한 스레드까지 폭넓은 내용을 배우고 탐구할 수 있는 유익한 강의였습니다.

5.2 프로젝트 수행 후기

주제를 정하고 구현을 시작할 당시에는 소켓과 스레드가 무엇인지, 실시간 통신이 어떤 구조로 이루어지는지, XOR 암호화와 대칭키·비대칭키의 차이는 무엇인지 전혀 모르는 상태였습니다. 그것들을 하나씩 공부하면서 기능을 붙여나가는 과정이 재미있었지만, 동시에 아쉬운 부분도 꽤 남는 프로젝트이기도 했습니다. 처음에는 완전한 P2P 구조를 목표로 했는데, 현재까지 배운 내용만으로는 한계에 부딪혀 이 형태로 마무리하게 됐습니다. IP를 전화번호처럼 변환해 입력받는 방식도 시도해봤지만 중간 자릿수가 잘리는 문제가 있어 포기해야 했습니다. 현재는 같은 네트워크 내에서만 연결이 가능한데, 언젠가 완전한 P2P를 구현하게 된다면 이 제한도 없애고 싶습니다. 포트 번호를 사용자가 직접 설정하는 기능도 구상했지만 실제 구현까지 이어지지는 못했습니다. 추후 관련 강의를 더 들으면서 실력을 쌓게 되면, 이 프로젝트를 다시 꺼내 완성하고 싶다는 생각이 들었습니다. 암호화 방식에 대해서도 아쉬운 부분이 있습니다. 처음에는 비대칭키 방식으로 구현하려 했는데 난이도가 너무 높아져서 결국 대칭 키 방식으로 진행하게 됐습니다. 대칭키는 구현이 간단하다는 장점이 있지만, 암호화에 쓰이는 시드 파일을 채팅 전에 양쪽이 미리 똑같이 가지고 있어야 한다는 단점이 있고 지금은 시드 파일을 직접 전달해야 하는데, 나중에 실력이 더 쌓이면 비대칭키 방식으로 바뀌서 시드 파일 없이도 안전하게 키를 주고받을 수 있도록 개선하고 싶습니다 하지만 아쉬운 점만 남은 것은 아닙니다. 수업에서 배운 내용들을 직접 코드에 적용해보면서 자바 기초 프로그래밍 실력, OOP 이해, 암호화·복호화, 소켓과 스레드에 대한 지식을 두루 쌓을 수 있는 의미 있는 프로젝트였다고 생각합니다.